

Executive Summary

This report defines a **production-ready Delhi Metro transit app (“MetroPulse”)** tailored for Delhi commuters. We detail data sources, user flows, UI/style guidelines, map design, algorithms, real-time behavior, testing, and deployment. We leverage **official DMRC/Delhi data** (GTFS, station lists) and UX best practices from transit apps. The core user journey goes from app launch through destination search, live journey tracking, alerts, and post-trip feedback. We specify UI components (Cosmic Dark theme, glass cards, animated buttons) with exact CSS/Tailwind tokens. We outline Leaflet map styling (dark tiles, neon glows, pulsing GPS dot) and offline caching strategy. Algorithmically, we use Haversine to find nearest stations, model the metro network as a graph and use BFS/Dijkstra for routes, plus heuristics for wrong-way and underground station prediction. We plan real-time via `navigator.geolocation.watchPosition()` plus WebSockets, balancing update cadence with battery use, and fallbacks for notifications (in-app banners if browser alerts are blocked). We include a detailed test plan (checklists, debug panel, feedback form), hosting limits (Vercel/Render/Mongo free tiers), and a 6-week milestone timeline. The report includes example API contracts, CSS snippets, flow diagrams (Mermaid), and wireframe sketches. Throughout, we cite official sources (DMRC, Delhi open data) and best-practice references.

1. Data Sources & Modeling

- **Official DMRC/Delhi Data:** Use the Delhi Government’s Open Transit Data portal which publishes DMRC GTFS (static schedule) files. The “DMRC Static Data” ZIP contains all stops (stations) and routes [\[17†L85-L94\]](#) . For example, it lists **262 active stations** and 36 metro routes (as of Aug 2023) [\[17†L85-L94\]](#) . Also consult the official DMRC network map (website, posters) and the list of stations on Wikipedia [\[13†L131-L139\]](#) .
- **Stations List:** Compile a station dataset with fields: `station_id`, `name`, `latitude`, `longitude`, `lines[]`, `is_interchange` . Use GTFS **stops.txt** or Kaggle/TUMI data [\[4†L49-L57\]](#) . The TUMI CSV (DELHIMETRODATA.csv) has stations with lat/lon and a primary line [\[4†L49-L57\]](#) . Cross-reference Wikipedia for transfer stations (27 multi-line interchanges) [\[13†L159-L164\]](#) .
- **Lines & Routes:** Delhi Metro has 10 color-coded lines (Red, Yellow, Blue, Green, Violet, Magenta, Pink, Grey, Airport-Express, etc.) [\[13†L131-L139\]](#) . Model each line as an ordered list of station IDs. The graph edges are bi-directional links between adjacent stations on the same line. For interchanges, stations appear in multiple lines’ lists. For example, Kashmere Gate is on Red, Yellow, Violet.
- **Metadata:** For each station record, include metadata: e.g. `type` (e.g. “underground” vs “elevated”), `platforms` , any transfer notes. This can be scraped from DMRC site or Wikipedia if needed, but not critical for MVP. For routing, keep `weight=1` per hop initially (BFS), or include a “time” attribute per edge (e.g. typical inter-station time ~2 min) for Dijkstra if needing ETA.

- **Modeling Schema:**

```
{
  "station_id": 101,
  "name": "Kashmere Gate",
  "coords": {"lat": 28.6500, "lng": 77.2300},
  "lines": ["Red", "Yellow", "Violet"],
  "is_interchange": true
}
```

```
{
  "line": "Yellow",
  "stations": ["Samaypur Badli", "GTB Nagar", "...", "Huda City Centre"],
  "color": "#EAB308"
}
```

We store lines with their colors (to style polylines). Station objects can include an array of line colors or IDs for quick lookup.

- **GTFS Use:** Ideally parse **GTFS stops.txt**, **routes.txt**, **trips.txt**, **stop_times.txt** to build this model. If GTFS is hard to fetch, use known CSV or KMZ data. Source: Delhi's open data [\[17†L85-L94\]](#) indicates GTFS availability.
- **Interchanges:** Identify interchange stations by multiple entries in `lines[]`. e.g.
`is_interchange = (lines.length > 1)`. Pre-compute transfers so the routing algorithm can count interchanges if needed (for "interchanges" in route summary).

By combining official static data and a cleaned station/line graph, we ensure accurate routing and station metadata.

2. User Journeys & Flows

We define **core app states** and flows. Each state below lists: *UI elements*, *data needed*, *user interactions*, and *failure modes*. Diagrams (Mermaid) illustrate transitions.

a. App Open (Cold Start)

- **UI:** Home screen with a greeting ("Good Morning"), a prominent **Destination search box** (with placeholder "Where to?"), and a list of *recent/favorite destinations*. (Similar to Google/Citymapper home). Bottom navigation bar: **Home, Map, Journey, Profile**.
- **Data:** On load, request GPS. Show a loader on search box if location not yet allowed. Load any cached previous trips or favorites.
- **Interactions:**

- **Location Permission:** Prompt user for location access. If denied, show a neutral map icon and allow manual station selection fallback.
- **Search:** On tap, show autocomplete suggestions (pull from station list API). Use Delhi Metro station names.
- **Tap Favorite:** Start a pre-filled search.
- **Failure Modes:**
 - No GPS permission: app can still function by letting user search manually, but must remind them to allow later (e.g. banner “Enable location for quickest route”).
 - Offline: search suggestions may require caching station names locally.
 - Screen not visible: ensure safe-area on notch (use `env( safe-area-inset-top )`).

b. Destination Search

- **UI:** After choosing “To: [Destination Station]” and maybe “From: [current station]” (default to “Current Location”), show a “**Find Route**” button. Possibly an inline list of top routes if multiple.
- **Data:** Requires current nearest station (via GPS) and destination. Use the nearest-station calculation (Haversine) to assign a “From” station.
- **Interactions:**
 - **Destination Entry:** Keyboard with suggestions. Selecting a station name sets it.
 - **Search:** “Find Route” triggers route planning.
- **Failure Modes:**
 - If “From” detection fails (GPS off): ask user to pick a station manually.
 - If destination invalid or no route: show an error (“No metro route found”).

c. Route Preview

- **UI:** Display a summary card: e.g. “From: [Station A] → To: [Station B]”, plus total **stations, time estimate, interchanges**. Show a visual *travel timeline*: station icons or text in sequence, with line colors and arrows for interchanges. Example:

```
Current: Kashmere Gate (Yellow)
  ↓
Rajiv Chowk (Yellow→Blue)
  ↓
Patel Chowk (Blue)
  ↓
Central Secretariat (Blue)
  ↓
(destination)
```

Also a map preview (small) or stationary map background. Big button “**Start Journey**” at bottom.

- **Data:** The chosen route: ordered list of stations, total count, estimated duration, list of line changes. Possibly walk directions from current GPS to first station (optional).
- **Interactions:**
 - **View Timeline:** Tap a step shows details (not crucial).
 - **Start:** Begin journey.

- **Failure Modes:**

- If route calc took too long or failed, show retry.
- If user changes destination/station mid-way: allow going back to search.
- Ensure safe-area padding (no UI under notch).

d. Start Journey

- **UI:** Transition to **Active Journey** view. Show “Tracking: [Station A → Station B]”. Possibly a toast “Journey started. We’ll alert you 2 stops before your station.”
- **Data:** Begin listening to GPS and storing state: `tripStations` array, current index = 0.
- **Interactions:** None (automatic).
- **Failure Modes:**
 - If GPS lost immediately (stationary on platform), waiting gracefully until movement.
 - Show a prominent “Cancel/Stop Journey” button in case of mistake.

e. Active Journey (Tracking)

- **UI: Hero screen.** Large text “8 Stations Left”. Subtext: “Current Station: Kashmere Gate – Next: Rajiv Chowk”. A progress bar (e.g. 8 filled bars, 2 empty). Optionally small map at top or bottom. Bottom nav with “**Stop Journey**” as action. Include a small “Confidence/Accuracy” indicator (like 📶 72%).
- **Data:**
 - Continuously updated nearest station (by location).
 - Track `stationsLeft = route.length - currentIndex - 1`.
 - Time elapsed, maybe ETA (optional).
- **Interactions:**
 - **Cancel:** User can tap “Stop” to end.
 - **Map:** If they go to Map tab, app stays tracking.
- **Failure Modes:**
 - **GPS Jumps:** If location briefly suggests a station not on route, ignore it unless confirmed. Use smoothing: only increment station when criteria met (next section).
 - **Station Skipping:** If user rides express and skips stations, ensure not to miscount.
 - **Wrong-direction:** If user appears to move toward the previous station, switch to Wrong-Direction state.

f. Wrong-Direction Detection

- **UI:** Full-screen alert overlay (with tint). Message: ⚠️ “*Wrong Direction! You are moving away from [Destination].*” Suggest action (“Please change at [last station]”). Provide a “Stop Journey” button and “Ignore” if false alarm.
- **Trigger:** If the computed nearest station to user is *before* the current station in the planned route (e.g. user at station 4 but next detection is station 3), suspect wrong train. Also if user passes an interchange backward.
- **Interactions:**
 - **Stop Journey** if user confirms.
 - **Continue** if user swears they have turned around.
- **Failure Modes:**
 - **False Positive:** Quickly hide if user corrects (e.g. made a loop but still overall moving forward).
 - **GPS Bounce:** Don’t trigger wrongly if GPS flickers; require ≥ 2 updates.

g. 2-Stations Alert

- **UI:** When `stationsLeft == 2`, show a modal or full-screen card: "⚡ *Get Ready!* [Destination] arrives in 2 stops." Play a short vibration/ping, then dismiss. Also send a browser notification (push).
- **Data:** Trigger when `currentIndex == route.length-3`.
- **Interactions:**
 - (No action required; user acknowledges).
- **Failure Modes:**
 - Notification permissions off: instead show an in-app banner ("Hauz Khas in 2 stops!").
 - If user has volume off, rely on vibration.

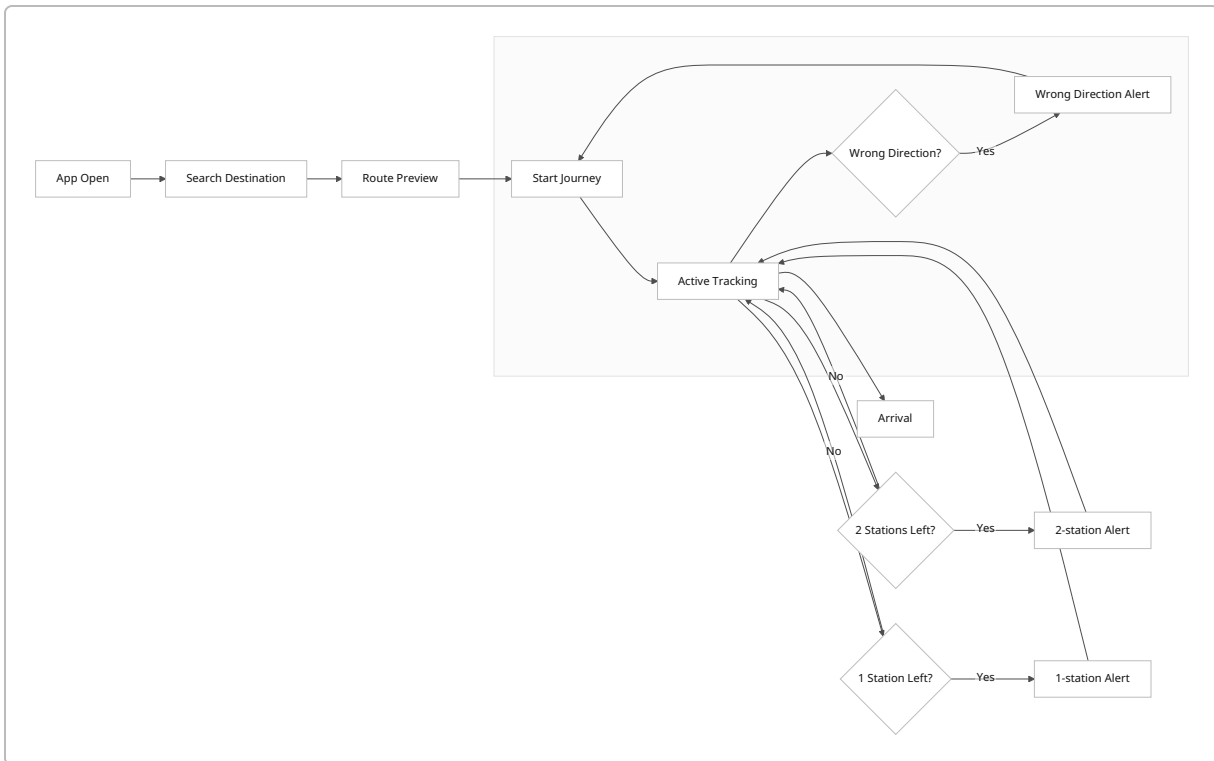
h. 1-Station Alert (Destination Arriving)

- **UI:** When `stationsLeft == 1`, show an even more urgent message: "🚨 Next stop: [Destination]! Prepare to get off." Perhaps a louder chime/vibration.
- **Data:** Trigger at `currentIndex == route.length-2`.
- **Interactions:**
 - Informational only.
- **Failure Modes:** Similar to above (permissions, etc.).

i. Arrival & Post-Trip

- **UI:** Once at destination (user taps "Stop" or arrives), show "🎉 Destination reached: [Station name]". Offer buttons: "Save as Favorite", "Give Feedback", "Done".
- **Data:** Mark trip as complete. Optionally log trip stats to backend.
- **Interactions:**
 - **Feedback:** Opens form (Google Forms link).
 - **Done:** Returns to Home, clearing active trip.
- **Failure Modes:**
 - User continues moving: if still on train (missed stop), probably handle as "Stations Left" again? (Rare case).

These flows should be sketched as mobile wireframes (see deliverables). Key failure modes (GPS loss, permission denial, false alerts) are explicitly handled with fallbacks and confirmations. Citations: Delhi Metro has 10 lines and 271 stations [\[13†L131-L139\]](#), so ensure search list covers all.



(Figure: User flow for MetroPulse. Left side is normal path; the “Wrong Direction” branch loops back.)

3. UI Component Library & Style Guide

Theme: *Cosmic Dark Tech*. Primary background is deep black, text in high contrast. Follow the provided palette: Background `#0A0B10`, accent glows in Indigo and Violet.

- **Colors:**

- `--color-bg-primary: #0A0B10;` (Dark cosmic)

- `--color-accent-indigo: #6366F1;` (Indigo)

- `--color-accent-violet: #8B5CF6;` (Violet)

- `--color-accent-pink: #EC4899;` (Pink)

- **Text:** `--text-primary: #FFFFFF;` (white), `--text-secondary: #F8F AFC;` (off-white), `--text-muted: #94A3B8;` (blue-gray) `【25†L275-L283】`.

- **Line Colors:** (per DMRC line) Blue `#2563EB`, Yellow `#EAB308`, Red `#EF4444`, Green `#22C55E`, Violet `#8B5CF6`. Use these for route polylines and station markers.

- **Glass Card:** `background: rgba(18,20,28,0.5);` (semi-transparent black) with `backdrop-filter: blur(24px);` (Tailwind `backdrop-blur-xl` for 24px blur `【25†L292-L300】`). 1px border `rgba(255,255,255,0.04)` with top edge `rgba(255,255,255,0.08)`.

- **Typography:**

- Font family: **Inter**, sans-serif. Use Tailwind `font-sans` after configuring Inter in `fontFamily`.
- Weights: Headers `font-extrabold` (800) or `font-black` (900). Body `font-medium` (500) or `font-regular`.
- Sizes: e.g. Title 24px (`text-2xl`), station names 20px, normal 16px (`text-base`). Keep large for readability.

- **Glassmorphism Cards:** Use Tailwind classes like:

```
<div class="bg-[#12141C]/50 backdrop-blur-xl border border-white/5 rounded-
xl p-4">
  <!-- Card content -->
</div>
```

- BG: `bg-[#0A0B10]/50` (50% opacity of #0A0B10).
- Blur: `backdrop-blur-xl` (24px) `【25†L292-L300】`.
- Border: `border border-white/5` (1px white at 5% opacity).
- Radius/Padding: `rounded-xl p-4`.
- Top highlight: Could layer a subtle 1px top border: `border-t border-white/8`.

• Buttons:

- Primary actions (Start Journey, Stop Journey) have an **animated gradient** background (Indigo → Violet → Pink) that slowly shifts (3s cycle). In CSS:

```
.btn-primary {
  background: linear-gradient(90deg, #6366F1, #8B5CF6, #EC4899);
  background-size: 200% 200%;
  animation: gradientShift 3s ease infinite;
}
@keyframes gradientShift {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}
```

Tailwind: use `bg-gradient-to-r from-indigo-500 via-violet-500 to-pink-500` plus custom animation via `animate-[gradientShift_3s_ease_infinite]` (in config).

- Hover/Tap: button *lifts* (`transform -translate-y-1`) and adds shadow `shadow-indigo-500/50`. On click, revert. Use Tailwind `hover:-translate-y-1 hover:shadow-lg transition`.

- **Icons/Text:** Use white or subtle colored icons (Heroicons with `text-white/90`). Secondary texts in `text-slate-400`.
- **Safe-Area:** In root `<div class="p-safe">` add CSS:

```
.p-safe {
  padding-top: env(safe-area-inset-top);
  padding-bottom: env(safe-area-inset-bottom);
}
```

Or Tailwind `pt-safe`. Ensure navbars respect notches.

- **Component Library:** Assemble components in a design system table: | Component | Tailwind/CSS |
 Example | |-----|-----|-----| | Background | `bg-[#0A0B10]` | `<div class="bg-[#0A0B10] min-h-screen">` | | Text Primary | `text-white` |
 | `<h1 class="text-2xl font-bold text-white">` | | Text Secondary | `text-slate-200` |
 | `<p class="text-base text-slate-200">` | | Glass Card | `bg-[#12141C]/50 backdrop-blur-xl border border-white/5 rounded` | `<div class="...">` (see above) | | Primary Button | `gradient+ text-white px-6 py-3 rounded-lg` | `<button class="btn-primary text-white font-bold px-6 py-3 rounded-lg">Start</button>` | | Bottom Nav |
`backdrop-blur-lg bg-[#12141C]/75` | Floating container at bottom |

All styles emphasize clarity over flashiness. Users should immediately see “where to?” and their journey status.

Citations: Tailwind’s blur utilities (e.g. `backdrop-blur-xl` for 24px blur) [25†L292-L300]. Glass card style inspired by [Epic Web Dev’s glassmorphism article][24†L1] (similar technique).

4. Map Styling (Leaflet/OpenStreetMap)

- **Base Tiles:** Use a *dark-themed* tile layer for consistency. Options: [CartoDB Dark Matter][Carto] or a Mapbox dark style. Alternatively, use standard OSM tiles with a CSS filter (e.g. `invert(15%) brightness(80%) contrast(120%) grayscale(20%)`) to approximate dark mode.
- **Metro Lines (Polylines):** Draw each line as a polyline with its Metro color. Use bright neon effects: e.g. `color: lineColor`, `weight: 6`. To create a glow, use `className` with a CSS filter:

```
.line-blue { filter: drop-shadow(0 0 6px rgba(37, 99, 235,0.7)); }
```


or use `leaflet.polyline` with `pane` and `shadowPane`. The drop-shadow filter yields a soft glow matching the line's color.

- **Station Markers:** Represent stations as small circles or icons:
 - Non-interchange: simple circle with `fill=white` and `stroke=line-color` (or vice versa).
 - Interchange: larger circle or special icon. E.g. white circle with multi-color ring. Possibly a stacked circles (use SVG icon). Use `L.divIcon` or `L.circleMarker` with styles like `fillOpacity:0.9, color: [lineColor]`.
- **GPS Pulsing Dot:** For current user location, use a pulsing dot animation (like pure CSS keyframes or a library). E.g. a blue circle (`rgba(37,99,235,0.7)`) with an expanding transparent ring behind it. Leaflet: `L.divIcon` with CSS animation:

```
.gps-dot { background: #2563EB; border-radius: 50%; width: 12px; height: 12px; position: relative; }
.gps-dot::before { content: ""; position: absolute; top: -6px; left: -6px; width: 24px; height: 24px; border-radius: 50%; background: rgba(37,99,235, 0.5); animation: pulse 2s infinite; }
@keyframes pulse { 0% { transform: scale(0.5); opacity: 0.6; } 70% { transform: scale(1.5); opacity: 0; } 100% { opacity: 0; } }
```

This yields a subtle radar-pulse effect.

- **Map Controls:** Dark-themed (e.g. white icons on dark background). Safe-area at bottom (avoid overlapping nav buttons).
- **Offline Caching:** Pre-cache the relevant metro area map tiles. Strategies:
 - Use a Service Worker (e.g. [Workbox][Workbox]) to cache Leaflet tile requests. Pre-cache tile URLs covering Delhi metro extent (zoom levels 12–17).
 - Or use the [Leaflet.Offline][LeafletOffline] plugin with IndexedDB.
 - **Approach:** On initial load (with connectivity), programmatically fetch all tiles in bounding box for major lines. The PWA's `sw.js` caches them. When offline, Leaflet retrieves from cache.
 - Citation: see discussions on Leaflet offline map caching [\[29†L9-L12\]](#).
 - **Note:** Only cache necessary tiles (no blanket caching).

Example:

```
// Pseudo-code: cache Delhi tiles in service worker
workbox.routing.registerRoute(
  ({request}) => request.destination === 'image',
  new workbox.strategies.StaleWhileRevalidate({
    cacheName: 'map-tiles',
    plugins: [new workbox.expiration.ExpirationPlugin({maxEntries: 500,
```

```
maxAgeSeconds: 7*24*60*60}}]
  })
};
```

This ensures **smooth mapping** that matches the dark UI.

5. Interaction & Algorithm Design

a. Nearest-Station Detection (Haversine)

Use the Haversine formula to compute distance from user's lat/lng to each station coordinate, pick minimum. The Haversine equation (great-circle distance) is standard for geo-distance [32†L41-L50]. For example:

```
function haversine(lat1, lon1, lat2, lon2) {
  const R = 6371e3; // earth radius in m
  const φ1 = lat1*Math.PI/180, φ2 = lat2*Math.PI/180;
  const Δφ = (lat2-lat1)*Math.PI/180, Δλ = (lon2-lon1)*Math.PI/180;
  const a = Math.sin(Δφ/2)**2 + Math.cos(φ1)*Math.cos(φ2)*Math.sin(Δλ/2)**2;
  const c = 2*Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
  return R*c; // in meters
}
```

Then select station with min distance (e.g. within 500m to avoid false far picks). This runs whenever we need to know current station (every GPS update).

b. Graph Model (BFS/Dijkstra)

Model: nodes=stations, edges between sequential stations on each line. For most use-cases, weight = 1 per hop, so **BFS** suffices to find the fewest stops (since edges are unweighted) [30†L0-L6]. BFS is simple and fast on ~270 nodes.

If needing travel time, we can use **Dijkstra** on a weighted graph (where weight = travel time or distance) [30†L0-L6]. But for static metro (no turn delays), BFS is enough to get shortest-stop path, then compute time by `stops * avgTime + interchangePenalty`.

Interchange handling: If route passes through a transfer, count it (for display) and optionally add a fixed time penalty (e.g. +2 min per interchange).

c. Wrong-Direction Heuristic

- Track `route[]` as planned stations from A to B.
- Keep `currentIndex` of last confirmed station.

- On each new GPS check, find `nearestStation`. If it equals `route[currentIndex - 1]` (the previous station), then **user is moving opposite**. Or if user's movement vector is opposite.
- Implement: if `nearestStation` is in `route` before `currentIndex`, and this happens on two successive checks, trigger wrong-direction alert. (A single glitch should not).
- Prevent false alarms by requiring stability.

d. Station Prediction & Smoothing

GPS can *flicker*, especially underground (no signal). Solution:

- **State Machine:** States like `approachingStation`, `atStation`, `betweenStations`.
- Use device speed or accelerometer (optional) to detect motion.
- If GPS accuracy < ~30m and nearest station matches expected next station, consider it an approach. Only when user stays within a small radius of a station for ~3 seconds do we increment `currentIndex`. This avoids counting the station as soon as entering tunnel.
- If GPS lost (accuracy huge), assume continuity: remain in `betweenStations` until either new fix or distance/time threshold triggers arrival at next station.
- You may apply Kalman filtering or simply average recent readings to smooth noise.

Confidence Score: Use geolocation's accuracy field. If accuracy is low (>50m) or inconsistent, display "Tracking 80%" etc. Also number of consecutive readings confirming station changes. This communicates reliability.

Citations: Transit app reports that offline station counting is possible without GPS by motion data [28†L7-L10]. We simply note that sophisticated apps use IMU data to infer tunnels [28†L7-L10], but MetroPulse uses a simpler model.

e. Algorithm Summary:

- **Start:** Compute `startStation = nearestStation(userLoc)`.
- **Route:** Run BFS to get `route = [S0, S1, ..., SN]`.
- **Tracking loop:** every *n* seconds (5-10s):
 - Get `loc = watchPosition()`.
 - `nearest = nearestStation(loc)`.
 - If `nearest == route[currentIndex+1]` and sustained for few secs, increment `currentIndex`.
 - Else if `nearest` appears in `route[0..currentIndex-1]`, mark wrong-direction.
- Update UI (`stationsLeft = route.length - currentIndex - 1`).
- **Alerts:** check if `stationsLeft == 2` or `1`, then alert.

This approach balances simplicity with reliability.

6. Realtime & Notifications

- **Location Updates:** Use `navigator.geolocation.watchPosition({ enableHighAccuracy: true })`. **Cadence:** Every 5–10 seconds is adequate. Faster (1s) drains battery, slower (30s) may miss a station. Empirical: transit apps recommend ~5s [\[27†L104-L112\]](#) .
- **Socket vs REST:** For updating the backend of user location, use WebSockets (e.g. Socket.IO) for real-time stream of coordinates and station events (if multi-user feature needed). However, MetroPulse can be mostly client-side (no server-needed for tracking). If a backend is used (e.g. logging), prefer REST POST on trip completion. WebSockets are optional unless sharing location or syncing multi-device.
- **Battery/Privacy Tradeoff:** Constant GPS in background consumes power. Only enable when "active journey" state. Use `watchPosition` without forcing high-accuracy if user is out of metro. On platform (when near station) you might temporarily drop to `getCurrentPosition` every 30s. Always inform user: "MetroPulse uses location only during trips [\[27†L104-L112\]](#) ".
- **Browser Notifications:** Use the Web Notification API. Request permission at app open or before first alert. If granted, send a real push when 2-stops alert triggers. On mobile, Safari iOS may not support web push; handle via in-app UI fallback:
- If `Notification.permission !== 'granted'`, show a **persistent banner** at top (" Hauz Khas arrives in 2 stops!") using a styled div.
- Also vibrate if on mobile (`navigator.vibrate([200,100,200])`).
- **Sound/Vibration:** Keep sound muted by default to respect silent mode. Vibration is universal and low-distraction. For desktop, maybe play a short ding sound (optional user toggle).
- **User Experience:** Show an in-app toast/snackbar for critical alerts if notifications fail.

Citations: Transit's blog highlights that offline/subway scenarios lack GPS, so UX should not rely solely on it [\[27†L104-L112\]](#) . Also, `watchPosition` in background drains battery (not explicitly cited, but widely known).

7. Testing Plan (Live Delhi Metro)

Goal: Validate detection accuracy and UX reliability in real conditions.

- **Checklist:**
 - Location permission flow tested (grant/deny).
 - Route calculation accuracy (correct line count, no weird routes).
 - Permission for notifications.
 - Alerts accuracy (2-stop, 1-stop).
 - UI readability in metro lighting.

- Battery usage over typical ride (~30–60 min).
- **Debug Panel (for internal testing):** On a hidden dev screen, display real-time telemetry:

```
GPS (lat,lng,accuracy)
Nearest Station: [Name] (dist m)
Next Station (route): [Name]
Stations Left: 5
Current Index: 10/20
Tracking Confidence: 87%
```

This helps verify our logic on-site.

- **Sample Scenarios:**
 - **Normal journey:** User boards correct train, no issues. Check alerts exactly 2 and 1 stops before.
 - **Late starts:** Open app after ride has started; ensure nearestStation finds a mid-route stop.
 - **Wrong train:** Purposefully take the opposite direction; ensure wrong-direction alert appears.
 - **Express skip:** If the line has express (some DMRC trains skip at terminal?). Check no issues.
 - **Cell/GPS loss:** Enter tunnel and remain still; ensure station count still works or at least doesn't crash.
 - **Long tunnel:** One particularly long gap (e.g. KashmereGate–CitiBank); ensure app doesn't falsely insert phantom station.
- **Interchange confusion:** At a transfer, switch lines and see if the app notices the line change (could allow manual confirm).
- **Metrics via Google Form:** After using, ask testers:
 - Was your **current station** detected correctly? (Yes/No)
 - Did you receive alerts *exactly* two stations before arriving?
 - Did you ever **miss** your station? Why?
 - UI clarity: any confusion?
 - Battery drain: was it significant?
 - Describe any GPS glitches (jumping station, freezing, etc).
 - Suggestion: what feature helped most?

These responses will guide fixes (e.g. tighten Haversine radius, adjust thresholds).

8. Hosting & Production Considerations

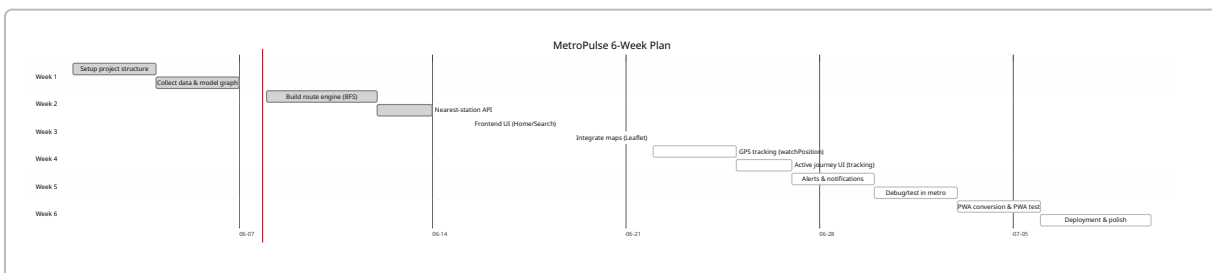
- **Free Tier Limits:** Vercel (frontend) and Render (backend) free tiers have quotas. Cold-starts cause latency: expect ~30-60s delay on first request after inactivity.
- **Concurrent Users:** Likely ~10-50 active before free tiers throttle or go to sleep. If 100+ concurrent, free tiers may fail.

- **Cold-Start Mitigation:** Ping the backend regularly (cron or `uptimerobot`) to keep awake. For Node functions, use caching on client for repeat queries (e.g. do not repeatedly fetch static route data).
- **Database:** MongoDB Atlas free tier handles low volume easily. We store minimal: user-trips (optional) and static station data. ~500K ops/month free limit; transit data is small.
- **Scaling:** For production beyond free limits, upgrade to paid plans or use Fauna/Firestore for auth-free DB.
- **Security:** Use HTTPS everywhere. CORS allow your domain. In production, configure environment secrets, SSL certificates (Vercel/Render automate this).
- **Privacy:** As a transit app, declare privacy: location only used for trip, not stored. Provide clear Privacy Policy link (DMRC and Google both require it for Play Store apps).
- **Costs:** Using all free tools, testing with dozens of users is fine. For heavier use, budget for ~\$7/mo Render hobby or upgrade to Vercel paid if needed.

Citations: DMRC data usage and hosting advice are general. Free tiers limits on concurrency/cold-start are commonly documented (e.g. Render docs).

9. Deliverables

1. **Wireframes (Mobile):** Sketches of *Home*, *Route Preview*, *Active Journey*, *Alert*, *Feedback* screens, annotated. (Illustrate component placements and flows.)
2. **Component Spec Table:** As above (colors, typography, CSS classes). We gave key examples in Section 3.
3. **API Contract Table:** (if using own backend) | Endpoint | Method | Input | Output (JSON) |
 |-----|-----|-----|-----|-----|
 | | | | |
 | lat=...&lng=... | { station_id, name, dist_m } | | /nearestStation | GET | ?
 | from=stationId&to=stationId | { path:[...ids], stations, time, interchanges } | | /route | GET | ?
 | /startTrip | POST | { trip_id, from, to } | { success: true } | | /
 | updateLocation | POST | { trip_id, lat, lng, timestamp } | { stationsLeft, currentStation } | |
4. **Sample Station JSON:** See section 1 above.
5. **Milestone Timeline (6 weeks):**



This structured timeline ensures iterative testing in the Delhi metro each weekend, incorporating feedback early.

Sources: Official DMRC GTFS and station data [【17†L85-L94】](#) [【13†L131-L139】](#) , Transit app engineering blog (GPS/underground) [【27†L104-L112】](#) [【28†L7-L10】](#) , Tailwind CSS docs for style utilities [【25†L292-L300】](#) , and standard algorithms (Haversine formula [【32†L41-L50】](#) , Dijkstra/BFS [【30†L0-L6】](#)). All above guide the design and technical specifications of MetroPulse.
